

Standard terminology character sets and encodings

Document #: P1859R0
Date: 2019-10-06
Project: Programming Language C++
SG16
EWG
CWG
Reply-to: Steve Downey
<sdowney@gmail.com, sdowney2@bloomberg.net>

Abstract: This document proposes new standard terms for the various encodings for character and string literals, and the encodings associated with some character types. It also proposes that the wording used for [lex.charset], [lex.ccon], [lex.string], and [basic.fundamental] 8 be modified to reflect the new terminology. This paper does not intend to propose any changes that would require changes in any currently conforming implementation.

§ 1 Introduction

In discussions around understanding the current capabilities of C++ and proposing new capabilities and facilities, SG16 has found that the current standard wording is often unclear, and does not match well the language currently used in 10646 and the Unicode Standard. This makes having technical discussions difficult. For example, the phrase “execution encoding” often comes up, or “presumed execution encoding”, trying to describe the encodings of char literals and strings as interpreted by the character classification functions. This conflates several concepts, and is not actually standard terminology. It would be useful to have standard terminology that did cover these concepts.

Execution character set is a standard term, however it defines what *abstract characters* must be included in the *character repertoire* of the character set used to encode C++, specifically the various kinds of character literals. That character set is a strict superset of the source character set, which defines the *abstract characters* must be in the *character repertoire* of the character set used to write C++ source code. The encodings of those character sets are not specified, and in fact there may be several encodings used depending on the context or kind of literal.

There are five encodings that are associated with the five kinds of character literals, corresponding to `char`, `wchar_t`, `char8_t`, `char16_t`, and `char32_t`. For 8, 16, and 32, the encodings must be UTF-8, UTF-16, and UTF-32. There are no associated encodings for `unsigned char` or `signed char`.

The encoding used for narrow and wide character and string literals is implementation defined, and is, of course, fixed at translation time.

At runtime, however, interpretation of character data is usually controlled by `locale`, either explicitly, or via the `locale` specified by `setlocale()`. The dynamic locale may not be the same as the literal encoding used at translation time. This is a source of errors in text processing.

Another source of problems is the baked in assumption that a single `wchar_t` can encode any representation character. For ABIs where `wchar_t` is 16 bits, this is not true, and many of the NTMBS functions are incomplete, as they do not allow for stateful wide character encodings.

§ 2 Terms

Literal Encoding

The encoding used for character and wide character and string literals in a translation unit.

Dynamic Encoding

The encoding implied by the `LC_CTYPE` category of `locale`.

Character Set [https://unicode.org/glossary/#character_set]

A collection of elements used to represent textual information.

Abstract Character [https://unicode.org/glossary/#abstract_character]

A unit of information used for the organization, control, or representation of textual data.

Character Repertoire [https://unicode.org/glossary/#character_repertoire]

The collection of characters included in a character set.

Basic source character set

The abstract characters that must be representable in the character set used for source code

Basic execution character set

The abstract characters the character repertoire of the character set used for literals must include. A superset of the abstract characters in the basic source character set.

Execution character set

The set of abstract characters representable by a `char` or `char` string literal

Execution wide-character set

The set of abstract characters representable by a `wchar_t` or `wchar_t` string literal

§ 3 Example of use (not an actual proposal, yet)

§ 3.1 Proposal Dnnnn

§ 3.1.1 `literal_encoding`

Returns an *unspecified* callable taking a range of elements of type `char` and returning a view of of code points decoded from the input range treating them as being in the *literal encoding* used for the current translation unit.

§ 3.1.2 `wide_literal_encoding`

Returns an *unspecified* callable taking a range of elements of type `char` and returning a view of of code points decoded from the input range treating them as being in the *wide literal encoding* used for the current translation unit.

§ 3.2 Discussion of proposal Dnnnn

Still woefully underspecified, it is at least clear what is being discussed, and how it might be something a compiler could implement. Without the terms *literal encoding* and *wide literal encoding* discussion gets bogged down quickly around the difference between what the compiler does and what locale and the *dynamic encoding* imply for character conversions.

§ 4 Wording

(lex.charset.1) The basic source character set consists of 96 abstract characters: the space character, the control characters representing horizontal tab, vertical tab, form feed, and new-line, plus the following 91 graphical characters:

```
a b c d e f g h i j k l m n o p q r s t u v w x y z
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
0 1 2 3 4 5 6 7 8 9
_ { } \[ \] # ( ) < > % : ; . ? * + - / ^ & | ~ ! = , \ " '

```

[Editorial Note: Should really be a list of unicode names or universal names, aka code points e.g. LATIN CAPITAL LETTER A LATIN CAPITAL LETTER B]

(lex.charset.3) The basic execution character set and the basic execution wide-character set shall each contain all the ~~members~~ abstract characters of the basic source character set, plus control characters representing alert, backspace, and carriage return, plus a null character (respectively, null wide character), whose value is 0. For each element in the basic execution character set, the encoded values of the members shall be non-negative and distinct from one another. In both the source and execution basic character sets, the value of each character after 0 in the above list of decimal digits shall be one greater than the

value of the previous. The execution character set and the execution wide-character set are implementation-defined supersets of the basic execution character set and the basic execution wide-character set, respectively. The encoded values of the members of the execution character sets ~~and the sets of additional members~~ are implementation defined~~locale-specific~~.

[lex.conn.2] A character literal that does not begin with u8, u, U, or L is an ordinary character literal. An ordinary character literal that contains a single c-char representable in the execution character set has type char, with value equal to the numerical value of ~~the encoding of~~ the c-char in the literal encoding. An ordinary character literal that contains more than one c-char is a multicharacter literal. A multicharacter literal, or an ordinary character literal containing a single c-char not representable in the execution character set, is conditionally-supported, has type int, and has an implementation-defined value.

[lex.conn.6] A character literal that begins with the letter L, such as L'z', is a wide-character literal. A wide-character literal has type wchar_t. The value of a wide-character literal containing a single c-char has value equal to the numerical value of the encoding of the c-char in the ~~execution wide-character set~~wide literal encoding, unless the c-char has no representation in the execution wide-character set, in which case the value is implementation-defined. ~~[Note: The type wchar_t is able to represent all members of the execution wide-character set (see [basic.fundamental]). —end note]~~ The value of a wide-character literal containing multiple c-chars is implementation-defined.